Conversion of HTML into JSP

Aim

Tounderstand JavaServer Pages (JSP) and convert the HTML page created in the previous Servlet experiment into a JSP page, while studying JSP architecture, lifecycle, JSP elements, directives, Expression Language (EL), and Servlet-JSP interaction.

Learning Objectives

• Understand JSP anditspurpose.

• Understand JSP architecture and lifecycle.

• Differentiate Servlet and JSP.

• Convert an existing HTML page into JSP.

• Understand JSP directives and Expression Language.

• Pass data from a Servlet to JSP and display it using EL.

• Understand JSP as the presentation layer in MVC.

1. Introduction to JSP

JSPstandsfor **JavaServer Pages**.It is a server-side web technology used to create dynamic web pages. A JSP page mainly contains HTML along with JSP elements, Expression Language (EL), JSTL and other tag-based features.
The key idea is that JSP allows HTML pages whose content can be generated dynamically on the server.

Why JSP?

```
Servlet:
out.println("<h1>Registration Successful</h1>");
out.println("<p>Name: " + name + "</p>");



JSP:
<h1>Registration Successful</h1>
<p>Name: ${name}</p>
```

Thus, **Servlet → processing/controller** and **JSP → presentation/view**.

2. Servlet vs JSP

Feature	Servlet	JSP
Form	Java class	JSP page
Focus	Request processing	Presentation/HTML
Typical role	Controller	View
HTML	Often generated using PrintWriter	Written directly
Dynamic data	Prepared in Java	Displayed using EL/JSTL

3. JSP Architecture

```
Browser
|
| HTTP Request
v

Tomcat
   |
   v
JSP Page
   |
   | Translation
```

```
   v
Servlet Source
   |
   | Compilation
   v
Servlet Class
   |
   | Execution
   v
HTML Response
   |
   v
 Browser
```

A JSP is translated into a Servlet by the JSP container, compiled and executed to produce the response.

4. JSP Lifecycle

```
JSP Page
   ↓
Translation
   ↓
Compilation
   ↓
Class Loading
   ↓
Instantiation
   ↓
jspInit()
   ↓
_jspService()
   ↓
jspDestroy()
```

• **jspInit()** — initialization.

• **_jspService()** — processes requests and generates the response; normally managed by the container. •

jspDestroy() — cleanup before the JSP is removed from service.

5. JSP Elements

Declaration:

```
<%! int count = 0; %>
```

Scriptlet:

```
<% String name = "Servlet"; %>
```

Expression:

```
<%= name %>
```

JSP Comment:

```
<%-- This comment is not sent to the browser --%>
```

Expression Language:

```
${name}
${student.name}
${param.name}
```

Scriptlets are shown for syllabus understanding; modern applications generally prefer EL, JSTL and Java-side components for presentation logic.

6. JSP Directives

Page:

```
<%@ page contentType="text/html;charset=UTF-8" %>
```

Include:

```
<%@ include file="header.jsp" %>
```

Taglib:

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

7. Expression Language (EL)

ELprovidesasimplewaytoaccessvaluesstored in request, session, application and other scopes.

```
${name}
${student.name}
${student.email}
${param.name}
```

8. Practical — Convert HTML File into JSP

Step 1: Previous HTML file

```
<!DOCTYPE html>
<html>
<head><title>Student    Information</title></head>
<body>
<h1>Student Information</h1>
<form action="hello" method="post">
Name:   <input   type="text"   name="name"><br><br>
Email:  <input  type="email"  name="email"><br><br>
Course: <input type="text" name="course"><br><br>
Age:   <input   type="number"   name="age"><br><br>
<input type="submit" value="Register">
</form>
</body>
</html>
```

Step 2: Rename index.html to student.jsp

Add the JSP page directive at the top. The HTML can remain almost unchanged.

```
<%@ page contentType="text/html;charset=UTF-8" %>

<!DOCTYPE html>
<html>
<head><title>Student    Information</title></head>
<body>
<h1>Student Information</h1>
<form action="hello" method="post">
Name:   <input   type="text"   name="name"><br><br>
Email:  <input  type="email"  name="email"><br><br>
Course: <input type="text" name="course"><br><br>
Age:   <input   type="number"   name="age"><br><br>
<input type="submit" value="Register">
</form>
</body>
</html>
```

9. Update the Servlet

```
package com.servlet;

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
        String age = request.getParameter("age");

        request.setAttribute("name", name);
        request.setAttribute("email", email);
        request.setAttribute("course", course);
        request.setAttribute("age", age);

         request.getRequestDispatcher("result.jsp")
                .forward(request, response);
```

```
    }
}
```

10. Create result.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>

<!DOCTYPE html>
<html>
<head><title>Registration  Result</title></head>
<body>
<h1>Registration Successful</h1>
<hr>
<h3>Student Details</h3>
<p>Name: ${name}</p>
<p>Email: ${email}</p>
<p>Course: ${course}</p>
<p>Age: ${age}</p>
</body>
</html>
```

11. Servlet → JSP Flow

```
Browser
   ↓
student.jsp
   ↓
Form submitted to /hello

HelloServlet
   ↓
getParameter(...)
   ↓
setAttribute(...)
   ↓
RequestDispatcher.forward(...)
   ↓
result.jsp
   ↓
HTML Response
   ↓
Browser
```

• **request.getParameter("name")** reads the form value whose name is "name".

• **request.setAttribute()** stores data in request scope.

• **forward()** forwards the request to result.jsp.

• **${name}** retrieves the request attribute through EL.

12. Project Structure

```
servlet/
■■■  pom.xml
■■■  src/
    ■■■  main/
        ■■■  java/
        ■    ■■■  com/servlet/HelloServlet.java
        ■■■  webapp/
            ■ ■ ■ student.jsp
            ■■■  result.jsp
```

13. Build and Deploy

```
mvn package
```

After a successful build, Maven creates **target/servlet.war**. Copy the WAR into Tomcat's **webapps** folder, start Tomcat, and open:

```
http://localhost:8080/servlet/student.jsp
```

14. Expected Output

First page: Student Information form with Name, Email, Course, Age and Register button.

After submission: Registration Successful followed by the entered student details.

15. Before and After Conversion

```
Before:
index.html
           → Servlet → PrintWriter → HTML Response

After:
student.jsp → Servlet → result.jsp → Browser
```

The mainchange is separation of responsibilities: the Servlet handles request processing, while JSP handles presentation.

16. JSP and MVC

```
Browser
↓



Servlet (Controller)
   ↓
Business Logic / Data Processing
   ↓
JSP (View)
   ↓
Browser
```

JSP does not replace the Servlet. They work together: Servlet commonly acts as controller and JSP as view.

17. Optional JSTL Example

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<c:if test="${not empty name}">
    <h1>Registration Successful</h1>
    <p>Name: ${name}</p>
    <p>Email: ${email}</p>
    <p>Course: ${course}</p>
    <p>Age: ${age}</p>
</c:if>
```

18. Practical Tasks

• Convert theexistingHTMLform into JSP.

• Add the page directive.

• Read all form parameters in the Servlet.

• Pass values using request.setAttribute().

• Forward to result.jsp.

• Display values using EL.

• Add one new form field and display it in result.jsp.

• Run and verify the complete request-response flow.

19. Viva Questions

Q: What does JSP stand for?
Ans: JavaServer Pages.

Q: Is JSP executed directly as HTML?
Ans: No. It is translated into a Servlet and executed by the container.
Q: What is the role of JSP?
Ans: Primarily presentation/view generation.

Q: What is the role of a Servlet?
Ans: Request processing/controller logic.

Q: What is EL?
Ans: Expression Language used to access values in JSP.

Q: What does request.setAttribute() do?
Ans: Stores data in request scope.

Q: What does forward() do?
Ans: Forwards the request to another server-side resource such as a JSP.

Q: What is jspInit()?
Ans: JSP initialization method.

Q: What is _jspService()?
Ans: Container-managed method that services JSP requests.

Q: What is jspDestroy()?
Ans: JSP cleanup method.

20. Key Points

• JSP is a server-side technology for dynamic web pages.

• Normal HTML can be written directly in JSP.

• JSP is translated into a Servlet by the JSP container.

• Servlets commonly process requests; JSP commonly handles presentation.

• EL simplifies display of server-side data.

• request.setAttribute() and forward() pass request data from Servlet to JSP.

• Servlet and JSP can work together in MVC.

• For Tomcat 11, use the Jakarta Servlet/JSP ecosystem rather than the older javax.servlet namespace.

Conclusion

The HTML page from the Servlet experiment was converted into JSP. The Servlet processes form data, stores values as request attributes and forwards the request to result.jsp. The JSP displays the values using EL, demonstrating the practical relationship between Servlet as controller and JSP as presentation layer.